DESENVOLVIMENTO MOBILE · MÓDULO 15 · TRILHA KOTLIN · NÍVEL INTERMEDIÁRIO

Publicação na Google Play

Build de release, assinatura, App Bundle, Play Console, faixas de teste, verificação de desenvolvedor, LGPD e CI/CD

Prof. Dr. Francisco dos Santos Viana

São Luís – Maranhão – Brasil · 2026

Objetivos do módulo

Ao final deste módulo você será capaz de

- Preparar um build de release otimizado e versionado.
- Assinar o app e entender a Play App Signing.
- Criar a conta e a ficha do app na Play Console, incluindo as declarações obrigatórias.
- Conduzir testes interno, fechado e aberto e cumprir a regra de 12 testadores.
- Lançar de forma gradual e acompanhar a saúde do app em produção.
- Atender à LGPD e automatizar a publicação com CI/CD.

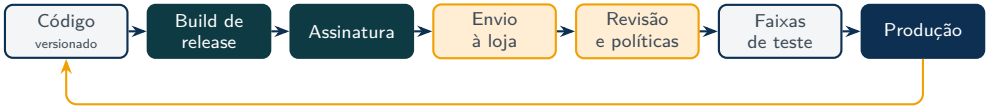
Roteiro

Neste módulo

- Fundamentos da distribuição de apps
- Preparando o release
- Assinatura do app
- Play Console
- Faixas de teste
- Lançamento e operação
- Automatizando com CI/CD
- Fechamento

Fundamentos da distribuição de apps

Do código ao aparelho do usuário



	Debug	Release
Objetivo	desenvolver	distribuir
Código	com verificações e depuração	otimizado, reduzido e ofuscável
Desempenho	mais lento	o real
Assinatura	chave de depuração automática	sua chave de release

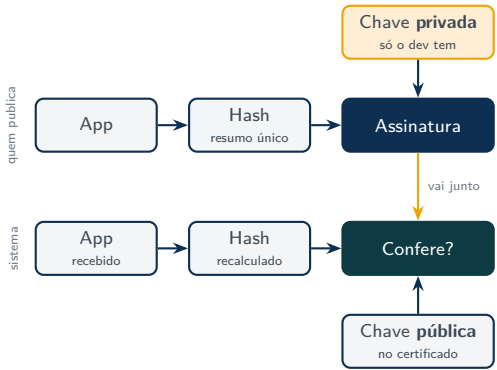
Build de release

Versão do app compilada para o usuário final: sem ferramentas de depuração, com código otimizado e assinada com a chave do desenvolvedor.

Atenção

Teste sempre o build de release antes de publicar: ofuscação e otimizações podem expor defeitos que não aparecem em debug.

Assinatura digital: a teoria



Assinatura digital

Resumo (*hash*) do pacote cifrado com a **chave privada** do desenvolvedor. Qualquer um verifica com a **chave pública**, mas só quem tem a privada consegue assinar.

O que ela garante

Integridade: o pacote não foi alterado.
Autenticidade: a atualização vem do mesmo autor do app instalado.

Analogia

O lacre de cera com o sinete do remetente: qualquer um reconhece o selo, ninguém o falsifica sem o anel.

Assinatura em cada plataforma

Android

O pacote é assinado com uma chave guardada num *keystore*. Com o **Play App Signing**, você assina com a **chave de upload**; o Google confere e reassina com a **chave do app**, que fica guardada com ele.



Dica

Perdeu a chave de upload? Com o Play App Signing, o Google permite trocá-la. Sem ele, perder a chave impede atualizar o app.

iOS

Todo app precisa ser assinado para rodar num iPhone. Três peças: o **certificado** (sua identidade), o **App ID** e o **perfil de provisionamento**, que liga certificado, app, capacidades e aparelhos.

Dica

O Xcode gerencia tudo isso com *Automatically manage signing*.

Atenção

Chaves e certificados são segredos: nunca no Git; guarde cópias num cofre de senhas.

Versionamento

Versionamento semântico (SemVer)

MAJOR.MINOR.PATCH: **MAJOR** muda com quebras de compatibilidade, **MINOR** com funcionalidades novas compatíveis e **PATCH** com correções.



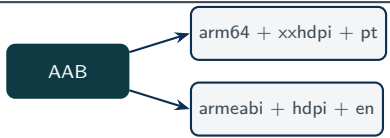
	Para pessoas	Para a loja
Android	`versionName`	`versionCode`
iOS	`CFBundleShort-VersionString`	`CFBundleVersion`
Flutter	`version: 2.4.1+27`	(o número após o `+`)

Regra das lojas

O número do build **sempre aumenta**. Não existe "voltar versão": para desfazer um problema, publica-se uma versão nova com número maior.

Formatos de pacote e tamanho do app

Formato	O que é
APK	pacote Android instalável, com tudo para todos os aparelhos
AAB	*Android App Bundle*: formato de **envio** à Play, que gera APKs sob medida para cada aparelho
IPA	pacote iOS enviado à App Store; a Apple também entrega variantes por aparelho (*App Thinning*)



Por que o tamanho importa

Apps grandes demoram a baixar, ocupam espaço e perdem instalações, principalmente em planos de dados limitados.

Como os builds encolhem

Tree shaking e ***shrinking*** removem código não usado; a **ofuscação** troca nomes por símbolos curtos; recursos são comprimidos e separados por aparelho.

Atenção

Código ofuscado gera relatórios de falha ilegíveis. Guarde os **símbolos de depuração** de cada versão para decodificá-los.

As duas grandes lojas

	Google Play	App Store
Conta	taxa única de US$ 25; pessoal ou organização	Apple Developer Program, US$ 99 por ano (isenção para instituições de ensino e sem fins lucrativos)
Revisão	automatizada e humana; de horas a dias (contas novas esperam mais)	humana (*App Review*); em geral de um a dois dias
Testes	faixas interna, fechada e aberta	TestFlight interno (até 100) e externo (até 10 000)
Lançamento gradual	percentual definido por você	*phased release* automático em 7 dias
Requisito anual	`targetSdk` 36 desde 31/08/2026	Xcode 26 e SDK do iOS 26 desde 28/04/2026
Comissão	de 15 % a 30 % sobre vendas digitais, conforme receita, produto e país	idem

Dica

Regras de comissão e de pagamento externo mudam com leis e decisões judiciais (União Europeia, EUA, Japão, Coreia). Confira antes de definir o modelo de negócio (Módulo 20).

Privacidade desde a concepção e LGPD

LGPD (Lei 13.709/2018)

Regula o tratamento de **dados pessoais** no Brasil: tudo que identifica ou pode identificar uma pessoa (nome, e-mail, matrícula, localização, identificador do aparelho).

Princípios que mais afetam um app

Finalidade (para quê), **necessidade** (o mínimo), **transparência** (explicar), **segurança** (proteger) e uma **base legal** para cada uso (consentimento, execução de contrato...).

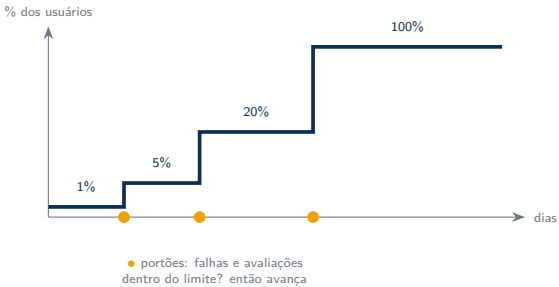
O que as lojas exigem

- Política de privacidade publicada.
- Declaração de dados coletados: *Data safety* (Play) e *App Privacy* (App Store).
- Textos de uso para cada permissão.
- **Exclusão de conta** dentro do app, se ele permite criar conta.

Atenção

A declaração deve incluir o que os SDKs de terceiros coletam (analytics, anúncios, Crashlytics). Não é assessoria jurídica: consulte o encarregado de dados (DPO) da instituição.

Gestão de lançamentos



Lançamento gradual

Liberar a versão para uma fração crescente dos usuários, avançando só se os indicadores de saúde continuarem bons.

Ferramentas do processo

- **Trem de versões**: lançar em datas fixas (a cada duas semanas, por exemplo).
- ***Feature flags***: ligar recursos remotamente, sem nova versão.
- **Versão mínima**: obrigar a atualização quando a antiga é insegura.
- **Notas de versão** claras para usuários e revisores.

ASO: o funil da loja



ASO — *App Store Optimization*

Conjunto de práticas para o app ser **encontrado** na busca da loja e **escolhido** por quem vê a ficha.

Etapa	O que influencia
Ser encontrado	título, subtítulo ou descrição curta, palavras-chave, categoria
Ser escolhido	ícone, capturas de tela, vídeo, nota e avaliações
Ser mantido	qualidade, desempenho, respostas às avaliações

Dica

As duas lojas permitem testes A/B da ficha (ícone, capturas, textos).

Depois do lançamento: saúde do app

Indicador	O que mede
Usuários sem falhas	% de usuários que não tiveram nenhum *crash* no período
Taxa de ANR	% de sessões em que a tela travou (Android)
Tempo de inicialização	do toque no ícone à primeira tela útil
Retenção D1/D7/D30	% de usuários que voltam 1, 7 e 30 dias após instalar
Nota e avaliações	percepção dos usuários; comentários apontam defeitos

Limites do Android vitals

Acima de 1,09 % de falhas percebidas ou 0,47 % de ANRs percebidos, o app é considerado de "mau comportamento" e perde visibilidade na Play.

Ciclo contínuo

Monitorar → priorizar → corrigir → lançar gradualmente → monitorar. Falhas e avaliações alimentam o *backlog* da próxima versão.

Dica

Responda às avaliações com educação e objetividade: a resposta é pública e mostra que o app é cuidado.

Preparando o release

Build de release

app/build.gradle.kts

```
android {
  defaultConfig {
    versionCode = 7          // inteiro, sempre crescente
    versionName = "1.2.0"    // exibido ao usuário
  }
  buildTypes {
    release {
      isMinifyEnabled = true      // R8: encolhe e ofusca
      isShrinkResources = true    // remove recursos não
      usados
      proguardFiles(getDefaultProguardFile(
          "proguard-android-optimize.txt"),
        "proguard-rules.pro")
      signingConfig = signingConfigs.getByName("release")
    }
  }
}
```

Versionamento

`versionCode` cresce a cada envio.
`versionName` segue o versionamento semântico (maior.menor.correção).

R8

Renomeia classes: bibliotecas com reflexão podem exigir regras `-keep`. **Teste o build release.**

Dica

Guarde o `mapping.txt` de cada versão: ele desofusca as pilhas de erro no Crashlytics e na Play Console.

Checklist antes de publicar

Produto

- Ícone adaptativo (*Image Asset Studio*) e nome definitivos.
- `applicationId` definitivo (não muda depois).
- `targetSdk` atendendo à exigência vigente.
- Textos em `strings.xml`; tema claro e escuro revisados.
- Mensagens de erro, estados vazios e modo offline testados.

Técnico

- Build release testado em aparelhos reais (versões antigas e novas).
- Logs de depuração e URLs de teste removidos.
- Nenhum segredo no código ou no repositório.
- Crashlytics ativo e `mapping.txt` enviado.
- Apenas as permissões realmente usadas.

Nível de API alvo

Desde **31/08/2026**, apps novos e atualizações na Google Play precisam de `targetSdk` **36** (Android 16) ou superior. A exigência sobe todo ano, por volta de agosto.

Assinatura do app

Criando a chave e assinando

Gerando a chave de upload

```
keytool -genkeypair -v -keystore upload.jks \
  -alias upload -keyalg RSA -keysize 2048 \
  -validity 10000
```

app/build.gradle.kts

```
signingConfigs {
  create("release") {
    storeFile = file(System.getenv("KEYSTORE_PATH")
      ?: "upload.jks")
    storePassword = System.getenv("KEYSTORE_PASS")
    keyAlias = "upload"
    keyPassword = System.getenv("KEY_PASS")
  }
}
```

Gerando o App Bundle

```
./gradlew bundleRelease
# app/build/outputs/bundle/release/app-release.aab
```

Nunca no Git

O `.jks` e as senhas **não** vão para o repositório. Use variáveis de ambiente, `~/.gradle/gradle.properties` ou *secrets* da CI.

Backup

Guarde o `.jks` e as senhas num gerenciador de senhas e em uma cópia offline.

Dica

Pelo Android Studio: **Build** → **Generate Signed App Bundle / APK** faz o mesmo com um assistente.

Play Console

A conta de desenvolvedor

	Pessoal	Organização
Para	pessoa física	empresas, instituições
Exige	verificação de identidade	D-U-N-S e verificação
Teste fechado obrigatório	sim (contas novas)	não
Nome exibido	seu nome	nome da organização

Custo

Taxa de registro **única**, paga ao criar a conta em `https://play.google.com/console`.

Para projetos institucionais

Apps que representam o IFMA, um laboratório ou um projeto de extensão ficam melhor em uma **conta de organização** da instituição, que garante continuidade quando os alunos se formam.

Atenção

A conta exige verificação de identidade, telefone e e-mail de contato públicos. Mantenha esses dados atualizados: contas inativas ou com dados inválidos podem ser encerradas.

Ficha da loja e declarações obrigatórias

Ficha da loja

- **Nome** do app (até 30 caracteres).
- **Descrição curta** (até 80) e **completa** (até 4000).
- **Ícone** 512×512 px.
- **Gráfico de recursos** 1024×500 px.
- **Capturas de tela** (celular e tablet).
- Categoria e e-mail de contato.

Conteúdo do app (Política)

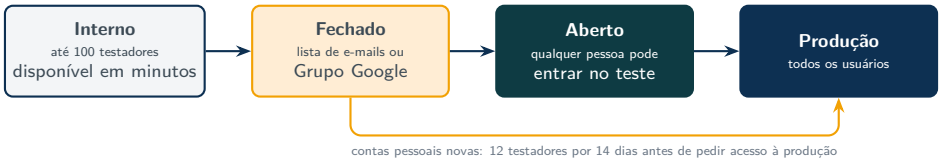
- URL da **política de privacidade**.
- Formulário de **segurança dos dados**.
- **Classificação de conteúdo** (questionário IARC).
- **Público-alvo** e se contém anúncios.
- Declaração de permissões sensíveis.
- Acesso para o revisor (se houver login).

Atenção

Se o app permite criar conta, ele deve oferecer **exclusão da conta** dentro do app e por um link na web.

Faixas de teste

Do teste interno à produção



Regra dos 12 testadores

Contas **pessoais** criadas após 13/11/2023 precisam manter um teste fechado com **pelo menos 12 testadores inscritos por 14 dias seguidos** antes de solicitar acesso à produção.

Na prática

- Recrute 15–20 testadores: se alguém sair, a contagem pode recomeçar.
- Uma turma do curso é um ótimo grupo de testadores!
- Publique atualizações durante o teste e registre o feedback.

Lançamento e operação

Na Play: lançamento gradual e operação

Lançamento gradual na prática

Ao promover a versão para produção, escolha a porcentagem inicial e aumente-a aos poucos, acompanhando o Android vitals. Problema? Interrompa o lançamento: quem ainda não atualizou continua na versão anterior.

Não existe "voltar versão"

Quem já atualizou fica com a versão nova. A correção é sempre outra versão, com `versionCode` maior.

Depois de publicado

- Responda às avaliações: a resposta é pública.
- Use **In-App Updates** para pedir a atualização de versões críticas.
- Use **In-App Review** para pedir avaliação no momento certo.
- Leia as notas de política da Play: as regras mudam.
- Planeje a atualização anual do `targetSdk`.

Distribuição fora da Play e verificação de desenvolvedor

Novidade de 2026 no Brasil

Desde **30/09/2026**, aparelhos Android certificados no Brasil (e também na Indonésia, em Singapura e na Tailândia) só instalam normalmente apps **registrados por desenvolvedores verificados**, inclusive quando distribuídos fora da Google Play (APK em site, lojas alternativas). A expansão global está prevista para 2027.

Como se registrar

Pela **Play Console** (apps na loja) ou pela **Android Developer Console** (apps só fora da loja), verificando a identidade e registrando o nome do pacote e a chave de assinatura.

Estudantes e hobbistas

Existem contas de **distribuição limitada**, pensadas para estudantes e projetos pessoais que instalam apps em poucos aparelhos.

O que não muda para o desenvolvimento

Instalar pelo Android Studio ou via **ADB** continua funcionando. Usuários avançados também têm um fluxo especial para instalar apps não verificados.

Dica

Para distribuir o LabMonitor só no laboratório via APK, registre o app antes.

Automatizando com CI/CD

Publicação automática

.github/workflows/release.yml (trecho)

```yaml
on:
  push:
    tags: ["v*"]          # ex.: git tag v1.2.0
jobs:
  release:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with: { distribution: temurin, java-version: 17 }
      - name: Restaurar keystore
        run: echo "$KEYSTORE_B64" | base64 -d > upload.jks
        env: { KEYSTORE_B64: "${{ secrets.KEYSTORE_B64 }}" }
      - name: Gerar AAB
        run: ./gradlew bundleRelease
        env:
          KEYSTORE_PASS: ${{ secrets.KEYSTORE_PASS }}
          KEY_PASS: ${{ secrets.KEY_PASS }}
      # enviar à faixa interna (Gradle Play Publisher
      # ou Fastlane supply) com uma conta de serviço
```

Pipeline típico

push de uma tag → testes → build assinado → envio à faixa interna → promoção manual para produção.

Ferramentas

Gradle Play Publisher (plugin Gradle) ou **Fastlane** (`supply`) usam a API da Play Console com uma *conta de serviço* do Google Cloud.

Atenção

Segredos ficam nos *secrets* do repositório, nunca no YAML.

Fechamento

Resumo

Resumo

- Release = R8, `versionCode` crescente, `targetSdk` atualizado e testes no build final.
- Play App Signing: você guarda a chave de upload; o Google guarda a chave do app.
- A loja exige ficha completa, política de privacidade, segurança dos dados e classificação.
- Faixas interna, fechada e aberta antes da produção; contas pessoais novas precisam de 12 testadores por 14 dias.
- No Brasil, desde 30/09/2026, apps precisam ser de desenvolvedores verificados mesmo fora da Play.
- LGPD: mínimo de dados, transparência, consentimento e exclusão. CI/CD automatiza tudo.

Glossário (1/2)

Termo	Significado
Build de release	versão otimizada e assinada do app, destinada ao usuário final
R8	encolhe, otimiza e ofusca o código no build de release
Assinatura digital	*hash* do pacote cifrado com a chave privada; prova integridade e autoria
Keystore	arquivo protegido por senha que guarda as chaves de assinatura
Play App Signing	o Google guarda a chave do app; você assina os envios com a chave de upload
AAB	*Android App Bundle*: formato de envio do qual a Play gera APKs por aparelho
`versionCode`	número interno que precisa crescer a cada versão enviada
`mapping.txt`	traduz pilhas ofuscadas pelo R8 de volta para nomes legíveis
SemVer	versionamento semântico do `versionName`: MAJOR.MINOR.PATCH

Glossário (2/2)

Termo	Significado
Faixa de teste	canal da Play para distribuir a grupos: interna, fechada ou aberta
Data safety	declaração dos dados coletados, exibida na ficha da loja
Lançamento gradual	liberação da versão para uma porcentagem crescente de usuários
Android vitals	falhas, ANRs e desempenho medidos nos aparelhos dos usuários
Verificação de desenvolvedor	registro de identidade exigido para instalar apps em aparelhos certificados
ASO	*App Store Optimization*: ser encontrado e escolhido na loja
CI/CD	compilar, testar e publicar automaticamente a cada mudança

Questões de revisão

1. O que a assinatura digital garante, e por que perder a chave pode impedir atualizações?
2. Qual a diferença entre a chave de upload e a chave do app no Play App Signing?
3. Por que o `versionCode` precisa sempre aumentar? Como "desfazer" uma versão com defeito?
4. Em que o AAB difere do APK, e que vantagem ele traz para o usuário?
5. Por que é preciso guardar o `mapping.txt` de cada versão?
6. Uma conta pessoal nova quer publicar. Que etapa de teste a Play exige antes da produção?
7. O que mudou, desde 30/09/2026, para distribuir um APK fora da Play no Brasil?
8. Como um lançamento gradual reduz o risco de uma versão nova?

Exercícios

Exercício 1 — Release

Gere o AAB assinado do LabMonitor, instale o build release com `bundletool` e verifique se tudo funciona com o R8 ativo.

Exercício 2 — Política

Escreva a política de privacidade do LabMonitor e preencha um rascunho do formulário de segurança dos dados com base no que o app realmente coleta.

Exercício 3 — Ficha

Produza ícone, gráfico de recursos, 4 capturas de tela e as descrições curta e completa do app.

LabMonitor — Etapa 15

Publique o app na faixa de **teste fechado** com a turma como testadores, colete o feedback por 14 dias e lance a versão 1.0.

Para aprofundar

- Preparar o app para lançamento: `https://developer.android.com/studio/publish/preparing`
- Assinatura de apps: `https://developer.android.com/studio/publish/app-signing`
- Requisitos de nível de API:
  `https://developer.android.com/google/play/requirements/target-sdk`
- Testes para contas pessoais novas:
  `https://support.google.com/googleplay/android-developer/answer/14151465`
- Verificação de desenvolvedor: `https://developer.android.com/developer-verification`
- LGPD (texto da lei):
  `https://www.planalto.gov.br/ccivil_03/_ato2015-2018/2018/lei/l13709.htm`

Respostas das questões de revisão

1. Integridade (o pacote não foi alterado) e autenticidade (vem do mesmo autor). O sistema só aceita atualizações assinadas pela mesma chave; sem ela, não há como provar a autoria.
2. A de upload assina o que você envia e pode ser trocada; a do app, guardada pelo Google, assina os APKs entregues aos aparelhos.
3. A loja e o sistema usam o `versionCode` para saber qual build é mais novo e recusam números menores. Para desfazer, publica-se uma versão nova, com número maior, contendo a correção.
4. O APK contém tudo para todos os aparelhos; do AAB a Play gera APKs só com o código, as imagens e os idiomas de cada aparelho: download menor.
5. O R8 renomeia classes e métodos; sem o mapeamento, as pilhas de falha no Android vitals e no Crashlytics ficam ilegíveis.
6. Um teste fechado com pelo menos 12 testadores inscritos por 14 dias seguidos, exigido de contas pessoais criadas depois de 13/11/2023.
7. Aparelhos certificados só instalam normalmente apps de desenvolvedores verificados, mesmo fora da Play: é preciso verificar a identidade e registrar o pacote e a chave.
8. Um defeito atinge só a fração inicial de usuários; os indicadores de saúde decidem se o lançamento avança ou é interrompido.

Fim do Módulo 15

Próximo: Módulo 16 (comum) — Backend terceirizado: Firebase, Supabase, AWS Amplify, Appwrite e outros BaaS do mercado.